PHASE ONE PRODUCTION PLAN

Boat Dealership Platform: $100K Budget Plan to Launch a Production-Ready Core

This report answers a narrower question than the full-platform valuation: with a hard starting budget of **$100,000**, what should be built now so the product can be live, secure, and operational, and what should be intentionally deferred until later.

BUDGET CAP	RECOMMENDED RELEASE TYPE	RECOMMENDED TIMELINE
$100,000	Production Pilot	10 Weeks

SHIP NOW

1 Core Wedge

Authenticated operating canvas with persistent graph editing, search, tree view, and exports.

DO NOT ATTEMPT NOW

Full Platform

The complete SaaS vision does not fit inside $100K without cutting production quality.

SAFE TARGET

1 Dealer Group

Launch for one internal organization or one pilot customer rather than broad enterprise rollout.

CRITICAL CONSTRAINT

Scope Discipline

Every non-essential feature removed now buys real production readiness later.

Executive Recommendation

A true full-platform build is not realistic at $100K. A **production-ready phase-one pilot** is realistic if the first release is defined tightly: one secure web product, one managed backend, one pilot organization, core persistence, basic roles, export, and deployment operations.

What Should Be Built Now

Build the current live diagram and builder into a secure hosted application where authenticated users can open the operating model, edit nodes and flows, save versions, search, inspect, export, and view the top-down flow map with persistent backend storage.

What Must Wait

Defer deep integrations, workflow automation, KPI engines, billing, enterprise multi-tenant expansion, advanced analytics, and broad customer deployment tooling. Those features belong in phase two and beyond.

```
$100K is enough for a production-ready core product only if the goal is "launch one narrow, secure, usable system" and not "finish the whole roadmap."
```

What Production-Ready Means at This Budget

Production-ready does not mean enterprise-complete. At this budget it should mean safe, hosted, backed up, authenticated, supportable, and usable by a real pilot team.

Must Exist

- HTTPS-hosted web app
- Authentication and basic roles
- Persistent database storage
- Graph save/load/version snapshot behavior
- Error monitoring and backups
- Basic admin access and support runbook

Can Stay Simple

- Single organization or limited pilot tenant model
- Manual user onboarding
- Managed authentication provider
- Managed database and hosting services
- Manual invoicing rather than self-serve billing

Cannot Be Fake

- Security basics
- Data persistence
- Role controls
- Backups and recovery
- Launch documentation
- Bug and error visibility

Recommended Phase-One Scope

The best use of the first $100K is to productionize the current interactive operating canvas as the initial product wedge. That keeps the existing prototype value, shortens time-to-launch, and avoids starting a new product from zero.

Phase-One Product Definition

- Secure login for pilot users
- One organization workspace with saved operating models
- Core live canvas with node and edge editing
- Persistent save, load, restore, and version snapshots
- Search, inspector, quick-focus modes, and top-down flow map
- Download Mermaid and JSON exports
- Browser-based PDF export support
- Basic user roles: admin and editor or viewer
- Cloud deployment, logging, monitoring, and backups

Recommended Delivery Shape

Treat this as a **production pilot release** for one real operator group, not a generic public launch. The product should be stable enough to run live, but intentionally narrow enough to fit inside budget.

Live canvas **Builder controls** **Tree view** **Version save** **Role-based access** **Export**

What Ships Now vs. What Waits

The line below is the most important decision in this entire plan. The only way to stay inside budget and still be production ready is to draw a hard boundary.

BUILD IN PHASE ONE	WAIT FOR LATER PHASES	REASON FOR DEFERRAL
Hosted app shell and secure login	Enterprise SSO, SCIM, broad identity features	Too costly for first launch; managed auth is enough for pilot production.
Persistent graph storage and version snapshots	Advanced collaboration, approvals, and workflow routing	Core save and restore matter now; collaboration workflow does not.
Node and edge editing with save/load	Live multiplayer editing and commenting	Single-user edit reliability is more important than concurrency in phase one.
Search, inspector, focus modes, tree view	Advanced KPI dashboards and embedded analytics	Users need navigation now; analytics can wait until real data exists.
JSON and Mermaid export	Full API platform and partner integrations	Export preserves portability; integrations are expensive and should follow product proof.
Basic admin roles and audit snapshots	Full enterprise RBAC matrix and compliance workflow	Admin/editor/viewer is enough for a pilot; deep policy layering is not.
Monitoring, backups, launch runbook	Billing engine, self-serve onboarding, subscription automation	Manual commercial ops is cheaper until the product has paying scale.
Browser PDF and operational exports	Server-side document generation suite	Current export behavior is already sufficient for launch and far cheaper to maintain.

Suggested Technical Approach

The cheapest way to get to production safely is to use managed services and reuse the current front-end interaction model instead of inventing a more ambitious stack.

Frontend

- Production web app built around the current live canvas and flow map behavior
- Keep search, focus modes, drag, save, export, and tree view
- Harden for responsive layout, accessibility, and error handling

Backend

- Managed Postgres + storage
- Managed auth provider for login and basic roles
- Simple API for graph save/load/version export
- One-organization workspace model first

Operations

- Managed hosting platform
- Error monitoring and uptime alerting
- Nightly backups or snapshot strategy
- Basic deployment pipeline and rollback path

```
Best-fit architecture at $100K = Managed Auth + Managed Database + Narrow API +
Productionized Existing Frontend
```

Budget Allocation: $100,000 Exactly

This budget only works if contingency is protected and low-value expansion is refused. The line items below deliberately bias toward getting one stable release live.

BUDGET AREA	AMOUNT	WHY IT IS INCLUDED
Product scoping, solution design, technical planning	**$7,000**	Prevents build drift and keeps the team inside the narrow launch scope.
UX cleanup, responsive polish, production interaction pass	**$6,000**	Turns the current prototype into something a pilot customer can actually use daily.
Frontend productionization of canvas, builder, search, tree view, exports	**$24,000**	The current UI is the product wedge, so this is the largest application line item.
Backend API, database schema, graph persistence, version snapshots	**$20,000**	Moves the app from browser-local state into real production persistence.
Authentication, user roles, workspace access control	**$10,000**	Production launch without auth and role control is not acceptable.
Admin surface, basic audit logging, restore flow	**$8,000**	Supports safe pilot operation and basic internal governance.
QA, accessibility, performance, acceptance testing	**$8,000**	Prevents a rushed launch from collapsing into support debt immediately.
Deployment, monitoring, logging, backups, launch hardening	**$7,000**	Ensures the product is actually supportable once live.
Documentation, training, project management, UAT	**$6,000**	Important for handoff, launch readiness, and a stable pilot operating motion.
Protected contingency reserve	**$4,000**	Small but necessary buffer for last-mile bugs and hosting surprises.
Total	**$100,000**	**Only valid if roadmap creep is actively rejected during delivery.**

Delivery Plan by Week

The delivery timeline below assumes a narrow team, managed services, and no major scope changes after planning week.

Week 1	Finalize scope, lock phase-one backlog, select managed services, define data model, and freeze launch criteria.
Weeks 2-3	Build application shell, auth, organization/workspace model, database schema, and initial graph persistence endpoints.
Weeks 4-5	Productionize live canvas behavior, node and edge editing, save/load flows, version snapshots, and export paths.
Weeks 6-7	Add tree-view connection to saved models, admin controls, restore path, search and inspector hardening, and role testing.
Week 8	Monitoring, backups, deployment pipeline, environment setup, and production hardening.
Week 9	QA, bug fixing, accessibility pass, UAT, and runbook completion.
Week 10	Pilot launch, controlled rollout to initial users, support monitoring, and immediate post-launch stabilization.

Launch Criteria

If the following criteria are not met, the product is not production ready yet, no matter how much of the UI feels done.

Required for Go-Live

- Authenticated access with at least admin and editor or viewer roles
- Persistent graph save and restore against the backend database
- Version snapshot or recovery path
- Functional search, inspector, and export flows
- Tree view reading the saved model
- Logging, error monitoring, and backup coverage
- Documented deploy and rollback steps

Not Required for Phase One

- Direct CRM, DMS, ERP, lender, marina, or telematics integrations
- Automated workflow engine and alert routing
- Embedded KPI warehouse and BI dashboards
- Subscription billing and self-serve signup
- Enterprise SSO, SCIM, and advanced permission matrices
- Real-time collaboration, comments, and multiplayer edit mode

Major Risks and Guardrails

The delivery risk is not that $100K is too small to build anything useful. The real risk is pretending it is enough to build everything useful.

Risk: Scope Explosion

Highest risk. New feature requests will consume the production budget before launch-quality work is finished.

- Guardrail: no new modules after week 1
- Guardrail: every add means a same-sized cut

Risk: Overbuilding Infrastructure

Trying to design a large enterprise platform too early will waste budget that should go to shipping the core product.

- Guardrail: use managed auth and database
- Guardrail: optimize for one pilot organization first

Risk: False Production Readiness

A polished UI without persistence, auth, backups, and support visibility is still not launchable.

- Guardrail: protect QA and operations budget
- Guardrail: launch checklist must be met before release

What Phase Two Should Fund Later

Once the product is live and used by a real pilot customer or internal operator team, the next dollars should go into depth and leverage, not redoing phase one.

Phase Two: Operational Depth

- Workflow automation
- KPI and reporting engine
- Approval flows and richer audit trails
- Template libraries and reusable playbooks

Phase Three: System Connectivity

- CRM, DMS, ERP, payment, and marina integrations
- Inbound and outbound data sync
- External API model
- Scheduled import and reconciliation logic

Phase Four: Scale Features

- True multi-tenant enterprise rollout
- Expanded RBAC and SSO
- Billing and subscription management
- Customer success tooling and rollout automation

Bottom-Line Decision

With **$100,000**, the right move is not to chase the full marine dealership platform. The right move is to launch a narrow, production-ready pilot centered on the current live operating canvas, persistent saved models, role-based access, exports, and one secure hosted workspace. That gets the product **up and running for real** while protecting the option to build the deeper platform later.

- Recommended immediate goal: production-ready pilot for one organization
- Recommended timeline: 10 weeks
- Recommended budget principle: protect core quality and defer everything not required for launch
- Recommended success measure: live users can securely save, edit, export, and manage the operating model in production